Quiz 1 Solutions

Question	Parts	Points
1: True or False	10	30
2: Graff Solutions Inc.	2	10
3: Combined Median	2	10
4: Big Cut	2	10
5: CRAZYSORT	3	15
6: Through Thick and Thin	2	15
7: One May Wonder	5	30
Total:		120

Name: _____

Problem 1. [30 points] **True or False** (10 parts)
Please fill in the bubble next to **True** or **False** for the following. *No justification is needed (nor will be considered).*

(a) [3 points] ◯ **True** ◯ **False**
There is a randomized algorithm that, when given $n \times n$ matrices A, B, C, checks whether or not $C = A+B$ in $O(n^2)$ time with the following guarantee: If $C = A+B$, then the algorithm outputs YES with probability 1; otherwise, the algorithm outputs NO with probability $\geq \frac{1}{2}$.

> **Solution:** True. One can simply check whether $C = A + B$ deterministically in $O(n^2)$ time by inspecting that $C_{i,j} = A_{i,j} + B_{i,j}$ for all $1 \leq i, j \leq n$.

(b) [3 points] ◯ **True** ◯ **False**
A Las Vegas algorithm with expected running time T can be turned into a Monte Carlo algorithm with running time $O(T)$ and success probability $\geq \frac{3}{4}$.

> **Solution:** True. We run the Las Vegas algorithm for $4T = O(T)$ time steps. If it stops and outputs the answer, we output that. Otherwise, we output something arbitrary and terminate. Since the expected running time of the Las Vegas algorithm is T, by Markov's inequality, the probability that it doesn't finish before $4T$ time steps is $\leq \frac{1}{4}$. Thus, with probability $\geq \frac{3}{4}$, we output the correct answer giving us a Monte Carlo algorithm with running time $O(T)$ and success probability $\geq \frac{3}{4}$.

(c) [3 points] ◯ **True** ◯ **False**
There exists a universal hash family $\mathcal{H} = \{h : \mathcal{U} \to M\}$ for $\mathcal{U} = \{0, \ldots, 127\}$, $M = \{0, 1\}$ with $|\mathcal{H}| = 6$.

> **Solution:** False. Let $\mathcal{H} = \{h_1, \ldots, h_6\}$. For any k, the number of possible sequences of values $(h_1(k), \ldots, h_6(k))$ is $2^6 < 2^7 = 128 = |\mathcal{U}|$, as $|M| = 2$. This means that there exist two keys $k_1, k_2 \in \mathcal{U}$ with $k_1 \neq k_2$ such that $h_i(k_1) = h_i(k_2)$ for all $1 \leq i \leq 6$. Hence $\mathcal{H}$ cannot be a universal hash family.

(d) [3 points] ◯ **True** ◯ **False**
Let $G = (V, E)$ be a finite, simple, connected, undirected, weighted graph with edge weights that are **not necessarily distinct**. Let $k \geq 2$ be an integer. Suppose that $e_1, \ldots, e_k$ form a set of k lightest edges in G. Additionally, suppose that all cycles in G are of length larger than k. Then, there is a minimum spanning tree of G that contains $e_1, \ldots, e_k$.

> **Solution:** True. Consider running Kruskal's algorithm on G. We first sort the edges of G by weight. Since $e_1, \ldots, e_k$ form a set of k lightest edges in G, we can sort the edges of G so that $e_1, \ldots, e_k$ are the first k edges. Since all cycles in G are of length larger than k, the edges $e_1, \ldots, e_k$ will not form any cycles. Thus, Kruskal's algorithm will add $e_1, \ldots, e_k$ to the minimum spanning tree and finally output a minimum spanning tree of G that contains $e_1, \ldots, e_k$.

(e) [3 points] ◯ **True** ◯ **False**

Let D be a data structure and let Φ_1 and Φ_2 be two valid potential functions that map the states of D to real numbers. We know that when using either one of Φ_1 and Φ_2 as the potential function, the amortized cost of D can be shown to be $O(1)$.

Define $\Phi = \Phi_1 + \Phi_2$, where $\Phi(D_i) = \Phi_1(D_i) + \Phi_2(D_i)$ for any state D_i of D. Then, using Φ as the potential function, the amortized cost of D can be shown to be $O(1)$.

Solution: True. Let D_0 be the initial state of D. Since Φ_1 and Φ_2 are valid potential functions, for any state D_i of D,

$$\Phi(D_i) = \Phi_1(D_i) + \Phi_2(D_i) \geq \Phi_1(D_0) + \Phi_2(D_0) = \Phi(D_0)$$

Hence, Φ is a valid potential function. Furthermore,

$$\begin{aligned}
\hat{c}_i &= c_i + \Delta\Phi_i \\
&= c_i + \Delta\Phi_{1,i} + \Delta\Phi_{2,i} \\
&\leq c_i + 2 \cdot \max\{\Delta\Phi_{1,i}, \Delta\Phi_{2,i}\} \\
&\leq 2 \cdot \max\{c_i + \Delta\Phi_{1,i}, c_i + \Delta\Phi_{2,i}\}
\end{aligned}$$

Thus, the new amortized cost is at most twice either of the prior amortized costs showing that using Φ as a potential function, the amortized cost of D can be shown to be $O(1)$.

(f) [3 points] ◯ **True** ◯ **False**

You maintain a sorted (in increasing order) list of n^2 distinct integers **stored in an array** (i.e., the ith integer in the sorted list can be looked up in constant time). Suddenly, you receive a set of $n+1$ new and distinct integers to be added to the list! You can compute the median of the combined list of n^2+n+1 integers in $O(n)$ time.

Solution: True. Let L denote the initial list of n^2 distinct integers and L' denote the final combined list of n^2+n+1 integers. For any integer $x \in L$, let $\text{RANK}_L(x)$ denote the rank of x in L, and for any integer $y \in L'$, let $\text{RANK}_{L'}(y)$ denote the rank of y in L'. Consider $x \in L \subset L'$. It is easy to see that

$$\text{RANK}_{L'}(x) \in \{\text{RANK}_L(x), \text{RANK}_L(x)+1, \ldots, \text{RANK}_L(x)+n+1\}$$

We are interested in the $y \in L'$ such that $\text{RANK}_{L'}(y) = \frac{n^2+n+1}{2}$. Note that such a y can only be one of the new $n+1$ integers or an $x \in L$ with $\text{RANK}_L(x) \in \{\frac{n^2-n-1}{2}, \ldots, \frac{n^2+n+1}{2}\}$. There are only $n+2$ such integers x and we can create a list L'' of them in $O(n)$ time (looking each of them up in $O(1)$ time). Now, the median of L' is the same as the median of the list $(L' \setminus L) \cup L''$ of length $2n+3 = O(n)$, which can be computed in $O(n)$ time.

(g) [3 points] ◯ **True** ◯ **False**

Suppose we build a dictionary using Cuckoo hashing. We are presented with two possible choices for the pair of hash functions $h_1, h_2 : \mathcal{U} \to M = \{0, 1, \ldots, 9\}$. The evaluations of the functions for the keys $k_1, \ldots, k_6 \in \mathcal{U}$ is given below.

Choice A	k_1	k_2	k_3	k_4	k_5	k_6
h_1	0	1	2	3	4	5
h_2	1	2	0	4	5	3

Choice B	k_1	k_2	k_3	k_4	k_5	k_6
h_1	0	1	2	2	3	4
h_2	1	2	0	3	4	2

We say that the hash functions h_1, h_2 are *bad* for the keys $k_1, \ldots, k_6$ if there is *some ordering* of the keys such that inserting them in that order will trigger an infinite loop.

Exactly one of the choices A and B of the hash functions h_1, h_2 is bad for the keys $k_1, \ldots, k_6$.

Solution: True. It is easy to see that Choice B will trigger an infinite loop when trying to insert the keys $k_1, \ldots, k_6$ in that order. It is also easy to see that no matter the order in which we insert $k_1, \ldots, k_6$, Choice A will not see any collisions and all keys will be stored in the positions determined by h_1.

(h) [3 points] ◯ **True** ◯ **False**

Recall the deterministic median finding algorithm from lecture. Suppose we make the following changes to it:

- Divide the list into groups of size $\sqrt{n}$ instead of groups of size 5 (after padding the list as necessary to make its length n a perfect square).
- **Use MERGESORT** to find the medians of the each of the groups.
- Recursively find the median of the medians.

The resulting algorithm would still be a linear, i.e., $O(n)$ time algorithm.

Solution: False. The runtime recurrence now becomes

$$T(n) = T(\sqrt{n}) + T\left(\frac{3n}{4}\right) + \Theta(n \log n)$$

This is because sorting each group of size $\sqrt{n}$ using MERGESORT to find their medians takes $\Theta(\sqrt{n} \log \sqrt{n}) = \Theta(\sqrt{n} \log n)$ time, and so finding all the medians requires $\sqrt{n} \cdot \Theta(\sqrt{n} \log n) = \Theta(n \log n)$ time. Then, finding the median of the medians requires $T(\sqrt{n})$ time. Then, we see that the "median of medians" element x is $\frac{3}{4}$-balanced. We know that at least $\frac{\sqrt{n}}{2}$ group medians are $\leq x$. For each of those $\frac{\sqrt{n}}{2}$ medians, additionally $\frac{\sqrt{n}}{2}$ other elements in that group are $\leq x$. So at least $\frac{n}{4}$ elements are $\leq x$. Similarly, there are at least $\frac{n}{4}$ elements are $\geq x$. Thus, x is indeed $\frac{3}{4}$-balanced, and we recurse on a subproblem of size at most $\frac{3n}{4}$ which requires $T(\frac{3n}{4})$ time. We find the subproblem to recurse on via a linear scan of the list comparing each element with x, which takes $\Theta(n) = O(n \log n)$ time. This recurrence already has a $\Theta(n \log n)$ term and hence $T(n) = \Omega(n \log n) = \omega(n)$.

(i) [3 points] ◯ **True** ◯ **False**

Recall the deterministic median finding algorithm from lecture. Suppose we make the following changes to it:

- Divide the list into groups of size $\sqrt{n}$ instead of groups of size 5 (after padding the list as necessary to make its length n a perfect square).
- **Recursively** find the medians of the each of the groups.
- Recursively find the median of the medians.

The resulting algorithm would still be a linear, i.e., $O(n)$ time algorithm.

Solution: False. The runtime recurrence now becomes

$$T(n) = (\sqrt{n}+1)T(\sqrt{n}) + T\left(\frac{3n}{4}\right) + \Theta(n)$$

since finding the median of each of the $\sqrt{n}$ groups as well as the median of the $\sqrt{n}$ medians requires a total of $(\sqrt{n}+1)T(\sqrt{n})$ time. The median of medians element x is still $\frac{3}{4}$-balanced exactly as shown in (h). We know that at least $\frac{\sqrt{n}}{2}$ group medians are $\leq x$. For each of those $\frac{\sqrt{n}}{2}$ medians, additionally $\frac{\sqrt{n}}{2}$ other elements in that group are $\leq x$. So at least $\frac{n}{4}$ elements are $\leq x$. Similarly, there are at least $\frac{n}{4}$ elements are $\geq x$. Thus, x is indeed $\frac{3}{4}$-balanced, and we recurse on a subproblem of size at most $\frac{3n}{4}$ which requires $T(\frac{3n}{4})$ time. We find the subproblem to recurse on via a linear scan of the list comparing each element with x, which takes $\Theta(n)$ time. From this recurrence, it is not immediate that the algorithm is not a linear time algorithm. However, one can show that this recurrence solves to $T(n) = \omega(n)$.

We will show that $T(n) \geq cn\log n$ for some constant c using induction. First, note that $T(1) \geq 0 = c\log 1$. Let c be such that $T(2) \geq 2c$, i.e., $c \leq \frac{T(2)}{2}$. These will form our base case. Our inductive hypothesis will be that $T(k) \geq ck\log k$ for all $k < n$, where $n \geq 3$. For the inductive step, we have

$$\begin{aligned}
T(n) &= (\sqrt{n}+1)T(\sqrt{n}) + T\left(\frac{3n}{4}\right) + \Theta(n) \\
&\geq \sqrt{n}T(\sqrt{n}) + T\left(\frac{3n}{4}\right) \\
&\geq \sqrt{n}\cdot c\sqrt{n}\log\sqrt{n} + c\cdot\frac{3n}{4}\log\left(\frac{3n}{4}\right) \\
&= \frac{5cn\log n}{4} - \frac{3cn}{4}\cdot\log\left(\frac{4}{3}\right) \\
&\geq cn\log n
\end{aligned}$$

where the last inequality holds because

$$n \geq 3 > \left(\frac{4}{3}\right)^3 \approx 2.37 \implies \log n \geq 3\log\left(\frac{4}{3}\right)$$

Hence $T(n) = \Omega(n \log n) = \omega(n)$.

Alternative Solution: Another way to see that $T(n) = \omega(n)$ is as follows. Let the constant underlying $\Theta(n)$ be lower bounded by c, i.e., let

$$T(n) \geq (\sqrt{n}+1)T(\sqrt{n}) + T\left(\frac{3n}{4}\right) + cn$$

We have

$$\begin{aligned}
T(n) &\geq (\sqrt{n}+1)T(\sqrt{n}) + T\left(\frac{3n}{4}\right) + cn \\
&\geq \sqrt{n}T(\sqrt{n}) + cn \\
&\geq \sqrt{n}(\sqrt[4]{n}T(\sqrt[4]{n}) + c\sqrt{n}) + cn \\
&= \sqrt[\frac{4}{3}]{n}T(\sqrt[4]{n}) + 2cn \\
&\geq \sqrt[\frac{4}{3}]{n}(\sqrt[8]{n}T(\sqrt[8]{n}) + c\sqrt[4]{n}) + 2cn \\
&= \sqrt[\frac{8}{7}]{n}T(\sqrt[8]{n}) + 3cn \\
&\ldots
\end{aligned}$$

The recursion unrolls $\log \log n$ times before the subproblem size, i.e., what is inside the $T(\cdot)$ term, becomes a constant. With each unroll, we get an additional cn term, and so we will have $T(n) = \Omega(n \log \log n) = \omega(n)$.

(j) [3 points] ◯ **True** ◯ **False**

Let $G = (V, E)$ be a finite, simple, connected, undirected, weighted graph with edge weights that are **not necessarily distinct**. Let $|V| = n$ and $|E| = m$. Suppose that all edge weights belong to the set $\{1, 2, \ldots, m\}$. Then, a minimum spanning tree of G can be found in $O(m\alpha(n))$ time, where $\alpha(\cdot)$ is the Inverse Ackermann function.

Solution: True. Since the edge weights belong to the set $\{1, 2, \ldots, m\}$, we can sort the edges by their weight using a linear scan in $O(m)$ time. Once this is done, we can run Kruskal's algorithm which will only take $O(m\alpha(n))$ time, yielding a total runtime of $O(m\alpha(n))$.

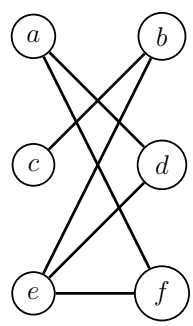
Problem 2. [10 points] **Graff Solutions Inc.** (2 parts)
Graff Solutions Inc. just opened a new department and hired several new employees to staff the department. Cole Auring is in charge of forming teams within the new department and assigning the new hires to the teams. As every team needs their own office, he wants to minimize the number of teams he forms.

To help with deciding team assignments, Cole decides to interview each new hire and identify incompatible pairs of them that can't be on the same team. He notes that the information he collects can be represented by a graph $G = (V, E)$, where the vertices in V represent new hires and the edges in E connect new hires that cannot work together. Cole sees that his task is now equivalent to partitioning V into the smallest number of groups possible such that no two vertices in the same group are connected by an edge.

Cole comes up with the following greedy algorithm to making team assignments. He interviews the new hires in some arbitrary order. Immediately after he interviews a new hire, he places them into the smallest existing team that they can work in. If there is no such existing team, Cole will create a new team and assign the new hire to it. Since Cole is lazy, once he assigns a new hire to a team, his decision is final.

The goal of this problem will be to show that Cole's algorithm is not competitive.

(a) [2 points] As a warm-up, consider the graph $G = (V, E)$ below, where $V = \{a, b, c, d, e, f\}$ is the set of new hires and the edges in E connect new hires that cannot work together.



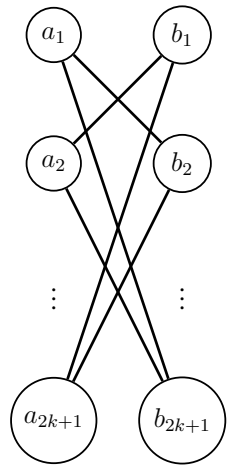
Suppose that Cole interviews the new hires in the order a, b, c, d, e, f. What is the competitive ratio of Cole's algorithm for this specific example, i.e., what is the ratio of the number of teams Cole forms to the number of teams in the optimal solution?

No justification is needed (nor will be considered).

Solution: $\frac{3}{2}$. Cole will end up creating a team for a, adding b to a's team, then creating a new team for c, adding d to c's team, and finally creating a new team for e, and adding f to the team with c and d. Thus, Cole will have three teams given by $\{a, b\}$, $\{c, d, f\}$, $\{e\}$. In the optimal solution, we only have two teams given by say $\{a, c, e\}$, $\{b, d, f\}$. Therefore, the competitive ratio of Cole's algorithm for this specific example is $\frac{3}{2}$.

(b) [8 points] Prove that Cole's algorithm is not k-competitive for any constant k.

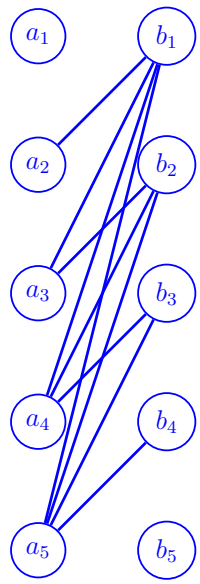
Solution: From part (a), it seems like bipartite graphs are a place to begin. Specifically, if we can set things up so that vertices on either side of a bipartite graph end up being assigned to the same team by Cole, Cole's solution will potentially be worse than optimal. Using this idea, for any $k > 0$, we construct the bipartite graph $G = (V, E)$ shown below, where $V = A \cup B$, $A = \{a_1, \ldots, a_{2k+1}\}$, $B = \{b_1, \ldots, b_{2k+1}\}$, and $E = \{(a_i, b_j) : i \neq j\}$.



Suppose Cole interviews the new hires in the order $a_1, b_1, a_2, b_2, \ldots, a_{2k+1}, b_{2k+1}$. Cole will end up creating a team for a_1, adding b_1 to a_1's team, then creating a new team for a_2, adding b_2 to a_2's team, ..., and finally creating a new team for a_{2k+1}, and adding b_{2k+1} to a_{2k+1}'s team. Thus, Cole will have $2k+1$ teams given by $\{a_1, b_1\}, \ldots, \{a_{2k+1}, b_{2k+1}\}$. In the optimal solution, we only have two teams given by say $\{a_1, \ldots, a_{2k+1}\}, \{b_1, \ldots, b_{2k+1}\}$. Therefore, the competitive ratio of Cole's algorithm for this specific example is $\frac{2k+1}{2} > k$. Therefore, Cole's algorithm is not k-competitive for any constant k.

Alternative Solutions:

- Another alternate solution is to generalize the bipartite graph to a complete t-partite graph of ℓ layers with the "horizontal" edges removed. This results in ℓ teams of size t and a competitive ratio of $\frac{\ell}{t}$. Picking $\ell > kt$ suffices.
- One common alternate solution is a bipartite graph with vertices a_i, b_j with edges between a_i and b_j if $i > j$, for $1 \leq i \leq 2k+1$. An example of this is shown below for the case of $k = 2$. Suppose Cole interviews the new hires in the order $a_1, b_1, a_2, b_2, \ldots, a_{2k+1}, b_{2k+1}$. Cole will end up creating a team for a_1, adding b_1 to a_1's team, then creating a new team for a_2, adding b_2 to a_2's team as it is currently the *smallest team* that they can work in, ..., and finally creating a new team for a_{2k+1}, and adding b_{2k+1} to a_{2k+1}'s team. As befores, Cole will have $2k+1$ teams. In the optimal solution, we only have two teams. Therefore, Cole's algorithm is not k-competitive for any constant k.



Common mistakes:

- Some students provided complete graphs. A complete graph would produce the same number of teams for any algorithm as all employees would need to be on separate teams, leading to a competitive ratio of 1.
- Some students provided complete bipartite graphs. A complete bipartite graph will produce only two teams in the greedy case, leading to a competitive ratio of 1. If we consider the natural partition of vertices into two sets A, B where edges only go between A and B, we can see that after we create two teams, one team must contain only vertices in A and the other vertices in B, and all vertices can be placed into exactly one team.
- Removing just one edge (or any fixed constant number of edges) from the complete bipartite graph results in a similar situation, except that our greedy algorithm produces at most three teams ($O(1)$ teams, respectively), leading to a competitive ratio of $O(1)$.

Problem 3. [10 points] **Combined Median** (2 parts)
Suppose you are given two lists A and B, each of length n and each sorted in increasing order. The $2n$ elements belonging to either of the lists are all **distinct**. Furthermore, each of the lists is **stored in an array** (i.e., the ith element of either list can be looked up in constant time).

The goal of this problem will be to find the *lower* median of the combined list of $2n$ elements.

(a) [2 points] Let $n = 9$ and let a_4 be the fourth element of A and b_5 be the fifth element of B. Suppose $a_4 < b_5$. Can a_4 be the *lower* median of the combined list of 18 elements? Briefly justify your answer.

Solution: No, a_4 cannot be the lower median of the combined list of 18 elements. This is because $a_4 < a_5, \ldots, a_9, b_5, \ldots, b_9$, i.e., a_4 is smaller than at least 10 elements in the combined list and hence its rank in the combined list is at most 8, while the lower median is the element of rank 9.

(b) [8 points] For general n, given A and B, design an $O(\log n)$ time algorithm to find the *lower* median of the combined list of $2n$ elements. Prove the correctness and runtime of your algorithm.

Solution: The idea, following from part (a), is to note that elements smaller than the medians of both lists (and symmetrically, larger than the medians of both lists) cannot be the median of the combined list. One has to be careful to handle the cases of even and odd and adapt the arguments using upper or lower medians as needed, but the core idea remains the same. This observation will let us trim our search space in half and so we can use a divide-and-conquer approach, essentially a binary search, to find the median of the combined list.

Let C denote the combined list of $2n$ elements. For any element $x \in A$, let $\text{RANK}_A(x)$ denote the rank of x in A, for any element $y \in B$, let $\text{RANK}_B(y)$ denote the rank of y in B, and for any element $z \in C$, let $\text{RANK}_C(z)$ denote the rank of z in C. We are interested in the $z \in C$ such that $\text{RANK}_C(z) = n$.

Case 1: $n = 2k - 1$ **is odd.** The medians of A and B are a_k and b_k respectively. If $x \in A$ with $\text{RANK}_A(x) < k$ and $x < b_k$, then $\text{RANK}_C(x) \leq 2k - 2$. Similarly, if $\text{RANK}_A(x) > k$ and $x > b_k$, then $\text{RANK}_C(x) \geq 2k + 1$.

Case 2: $n = 2k$ **is even.** The lower medians of A and B are a_k and b_k respectively. If $x \in A$ with $\text{RANK}_A(x) < k$ and $x < b_k$, then $\text{RANK}_C(x) \leq 2k - 2$. Similarly, if $\text{RANK}_A(x) > k$ and $x > b_k$, then $\text{RANK}_C(x) \geq 2k + 1$.

From the above, we have the following. Let m_A and m_B be the elements of A and B respectively of rank $\left\lceil \frac{n}{2} \right\rceil$ (these can be looked up in constant time). We are neither interested in the elements that are smaller than both m_A and m_B, nor the elements that are larger than both m_A and m_B. This means that:

- if $m_A < m_B$, we are interested in the elements $\geq m_A$ and $\leq m_B$, and
- if $m_A > m_B$, we are interested in the elements $\geq m_B$ and $\leq m_A$.

In either case, we can recurse on two lists A' and B' of length $\left\lfloor \frac{n}{2} \right\rfloor + 1$. In the case where $m_A < m_B$, $A' = \{x \in A : x \geq m_A\}$ and $B' = \{y \in B : y \leq m_B\}$. In the case where $m_A > m_B$, $A' = \{x \in A : x \leq m_A\}$ and $B' = \{y \in B : y \geq m_B\}$. Let C' denote the combined list of $2\left\lfloor \frac{n}{2} \right\rfloor + 2$ elements. Note that the element z we are after is the element of rank $n - \left(\left\lceil \frac{n}{2} \right\rceil - 1\right) = \left\lfloor \frac{n}{2} \right\rfloor + 1$ in C', i.e., the lower median of C'.
This gives us the following algorithm, $\textsc{Median}(A, i, B, j, n)$:

1. If $n \leq 2$, find and return the median of $\{a_i, \ldots, a_{i+n-1}, b_j, \ldots, b_{j+n-1}\}$.
2. Let $k = \left\lceil \frac{n}{2} \right\rceil$, $m_A = a_{i+k-1}$, and $m_B = b_{j+k-1}$.
3. If $m_A < m_B$, return $\textsc{Median}(A, i+k-1, B, j, \left\lfloor \frac{n}{2} \right\rfloor + 1)$.
4. If $m_A > m_B$, return $\textsc{Median}(A, i, B, j+k-1, \left\lfloor \frac{n}{2} \right\rfloor + 1)$.

Our algorithm simply runs $\textsc{Median}(A, 1, B, 1, n)$. The correctness of the algorithm follows from the previous arguments. The runtime of the algorithm is given by the recurrence relation

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) + O(1)$$

which solves to $T(n) = O(\log n)$.

Alternative Solution: The element we are searching for has rank n in the combined list, and this element belongs to either A or B. We can perform a binary search in A to find the element in A with rank n in the combined list, if it exists, and if not, perform a similar binary search in B. In order for an element a_i to have rank n in the combined list, we need to have $b_{n-i} < a_i < b_{n-i+1}$, as then a_i is greater than $i-1$ elements of A and $n-i$ elements of B, i.e., $n - i + (i-1) = n-1$ elements of the combined list. We can use this observation to perform a binary search for such an a_i using the following algorithm, the following algorithm, $\textsc{Search}(A, B, i, j, n)$:

1. If $i > j$, return FAIL.
2. Let $\text{MID} = \left\lfloor \frac{i+j}{2} \right\rfloor$.
3. If $b_{n-\text{MID}} < a_{\text{MID}} < b_{n-\text{MID}+1}$, return a_{MID}.
4. If $a_{\text{MID}} < b_{n-\text{MID}}$, return $\textsc{Search}(A, B, \text{MID}+1, j, n)$.
5. If $a_{\text{MID}} > b_{n-\text{MID}+1}$, return $\textsc{Search}(A, B, i, \text{MID}-1, n)$.

Our algorithm simply runs $\textsc{Search}(A, B, 1, n, n)$, and if it returns FAIL, it then runs $\textsc{Search}(B, A, 1, n, n)$. Notice from our argument above that exactly one of these two calls will succeed and return something other than FAIL. Since SEARCH is just a binary search, it runs in $O(\log n)$ time and hence our algorithm also runs in $O(\log n)$ time.

We now argue the correctness of the algorithm. Step 3 is clear from the argument from before. Let's look at Step 4. If $a_{\text{MID}} < b_{n-\text{MID}}$, this means that the rank of a_{MID} in the combined list is $< \text{MID} + n - \text{MID} = n$. This in particular means that no a_ℓ for any $\ell \leq \text{MID}$ can have a rank of n in the combined list, and therefore, we switch to search on $\ell \geq \text{MID} + 1$. Similarly, let's look at Step 5. If $a_{\text{MID}} > b_{n-\text{MID}+1}$, this means that the rank of a_{MID} in the combined list is $> \text{MID} + n - \text{MID} + 1 = n + 1$. This in particular means that no a_ℓ for any $\ell \geq \text{MID}$ can have a rank of n in the combined list, and therefore, we switch to search on $\ell \leq \text{MID} - 1$. Termination is ensured because in every call, either we return, or we make a recursive call to search in a *strictly* smaller interval.

Common mistakes:

- A lot of students followed the recommended solution, but did not delete the lower half of one list and the top half of the other list. Instead, they deleted only one. Although this is not incorrect by itself, as it depends on how one defined the recursive algorithm, if one uses the recursive algorithm MEDIAN, as we defined in the solution, that approach would be incorrect, as that function assumes that we have two lists of the same size, which is no longer true if one discards one half of only one of the two lists.
- Some students had the following algorithm. Suppose that the element of rank n in the combined list belongs to A (if not, repeat the following procedure assuming the element of rank n in the combined list belongs to B). First, search for the median of A in B. Based on the result of the search, if the median of A has rank n in the combined list, we're done. Otherwise, we learn the half of A that contains the element we're after. This gives a runtime recurrence of $T(n) = T\left(\frac{n}{2}\right) + \Theta(\log n)$, which solves to $\Theta(\log^2 n) = \omega(\log n)$.
- A lot of students forgot to mention a base case.

Problem 4. [10 points] **Big Cut** (2 parts)
Let $G = (V, E)$ be a finite, simple, undirected, and unweighted graph with $n = |V|$ vertices and $m = |E|$ edges. A *cut* of G is a partition $(S, V \setminus S)$ of the vertices into two sets.

The *value* of a cut $(S, V \setminus S)$ is equal to the number of edges crossing the partition, i.e.,

$$\textsc{val}(S) = |\{(u, v) \in E : u \in S, v \in V \setminus S\}|$$

(a) [4 points] Design an $O(n)$ time Monte Carlo algorithm which given G outputs a cut $(S, V \setminus S)$ of G with $\mathbb{E}[\textsc{val}(S)] \geq \frac{m}{2}$. Prove the correctness and runtime of your algorithm.

Solution: Our algorithm simply looks at each vertex and assigns it to either S or $V \setminus S$ independently and uniformly at random. This algorithm clearly runs in $O(n)$ time. We now prove that $\mathbb{E}[\textsc{val}(S)] \geq \frac{m}{2}$. For any edge $e \in E$, let X_e denote the indicator random variable corresponding to whether e crosses the partition. In order for e to cross the partition, its endpoints must be in different sides of the partition which happens with probability $2 \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{2}$. Therefore,

$$\mathbb{E}[X_e] = \Pr[X_e = 1] = \frac{1}{2}$$

Note that $\textsc{val}(S) = \sum_{e \in E} X_e$. By linearity of expectation,

$$\mathbb{E}[\textsc{val}(S)] = \sum_{e \in E} \mathbb{E}[X_e] = \frac{m}{2}$$

Alternative Solution: Another algorithm involves selecting $\lfloor \frac{n}{2} \rfloor$ vertices to be in S uniformly at random. Note that there are $\binom{n}{\lfloor \frac{n}{2} \rfloor}$ different possible sets S. For any edge e, there are $2\binom{n-2}{\lfloor \frac{n}{2} \rfloor - 1}$ sets S that will contain exactly one endpoint of e. Thus, the probability that e is a crossing edge is

$$\frac{2\binom{n-2}{\lfloor \frac{n}{2} \rfloor - 1}}{\binom{n}{\lfloor \frac{n}{2} \rfloor}} = \frac{2\left\lfloor \frac{n}{2} \right\rfloor \left(n - \left\lfloor \frac{n}{2} \right\rfloor\right)}{n(n-1)} > \frac{1}{2}$$

which we can show by considering the parity of n (i.e., $\lfloor \frac{n}{2} \rfloor$ is either $\frac{n}{2}$ or $\frac{n-1}{2}$). Similar to the earlier solution, defining the indicator X_e and applying the linearity of expectation, we get

$$\mathbb{E}[\textsc{val}(S)] = \sum_{e \in E} \mathbb{E}[X_e] > \frac{m}{2}$$

What remains to be argued is that selecting $\lfloor \frac{n}{2} \rfloor$ vertices can be done in $O(n)$ time. This is not as immediate to see. We provide two ways of doing this.

- **Method 1:** For each vertex $v \in V$, we sample a random value $x_v \in [0, 1]$. We then find the *lower* median y of the set $\{x_v : v \in V\}$ in $O(n)$ time using the median finding algorithm. Finally, we define

$$S = \{v : x_v \leq y\}$$

  Notice that assuming infinite precision in the sampling, for any two vertices v, v', $x_v \neq x'_v$. Therefore, $|S| = \lfloor \frac{n}{2} \rfloor$. We claim that S is a uniformly random subset of V of size $\lfloor \frac{n}{2} \rfloor$. To see why this is true, note that sampling random values for each vertex implictly induces a uniformly random permutation over the n vertices. There are $n!$ such permutations. Fix any subset of $T \subseteq V$ with $|T| = \lfloor \frac{n}{2} \rfloor$. We will calculate $\Pr[S = T]$. In order for $S = T$, the vertices in T must be the first $\lfloor \frac{n}{2} \rfloor$ in the random permutation induced by the random values that we sample. There are $\lfloor \frac{n}{2} \rfloor! \lceil \frac{n}{2} \rceil!$ such permutations, where there are $\lfloor \frac{n}{2} \rfloor!$ ways of permuting the vertices in T and $\lceil \frac{n}{2} \rceil!$ ways of permuting the vertices in $V \setminus T$. Therefore,

$$\Pr[S = T] = \frac{\lfloor \frac{n}{2} \rfloor! \lceil \frac{n}{2} \rceil!}{n!} = \frac{1}{\binom{n}{\lfloor \frac{n}{2} \rfloor}}$$

  as required.

- **Method 2:** Another approach is to repeatedly sample random vertices until we have $\lfloor \frac{n}{2} \rfloor$ of them. Care has to be taken, however, to ensure that we are able to do this in $O(n)$ time. To this end, suppose $V = \{v_1, \ldots, v_n\}$. We iterate through V. Suppose we are at v_i and we have added t vertices to S already. We then add v_i to S uniformly with probability $\frac{\lfloor \frac{n}{2} \rfloor - t}{n-i+1}$. We do this until we reach the end of V. This algorithm clearly runs in $O(n)$ time.
  We first claim that $|S| = \lfloor \frac{n}{2} \rfloor$ at the end. As a sanity check, note that once $t = |S| = \lfloor \frac{n}{2} \rfloor$, the probability $\frac{\lfloor \frac{n}{2} \rfloor - t}{n-i+1} = 0$ and thus no more vertices will be added to S. Therefore, $|S| \leq \lfloor \frac{n}{2} \rfloor$. Furthermore, suppose $|S| = \lfloor \frac{n}{2} \rfloor - k$, i.e., we need k more vertices in S, and $i = n - k + 1$, i.e., we only have k vertices remaining. Then, the probability $\frac{\lfloor \frac{n}{2} \rfloor - t}{n-i+1} = 1$ and thus all the remaining vertices will be added to S. Therefore, $|S| \geq \lfloor \frac{n}{2} \rfloor$. Combining these observations, we have $|S| = \lfloor \frac{n}{2} \rfloor$ at the end, as required.
  We next claim that S is a uniformly random subset of V of size $\lfloor \frac{n}{2} \rfloor$. To see why this is true, fix any subset of $T \subseteq V$ with $|T| = \lfloor \frac{n}{2} \rfloor$. We will calculate $\Pr[S = T]$. In order for $S = T$, the vertices in T must be the $\lfloor \frac{n}{2} \rfloor$ ones chosen.
  Suppose $T = \{v_{i_1}, \ldots, v_{i_{\lfloor \frac{n}{2} \rfloor}}\}$ and define $i_0 = 0$.

The probability that the vertices in T are the ones that are picked is

$$\prod_{j=1}^{\lfloor \frac{n}{2} \rfloor} \left[\left(\prod_{k=i_{j-1}+1}^{i_j - 1} \Pr[v_k \text{ was not added to } S] \right) \cdot \Pr[v_{i_j} \text{ was added to } S] \right]$$

Now, note that

$$\Pr[v_k \text{ was not added to } S] = 1 - \frac{\lfloor \frac{n}{2} \rfloor - (j-1)}{n-k+1}$$

as $j-1$ vertices have been added to S when we reach v_k. Similarly,

$$\Pr[v_{i_j} \text{ was added to } S] = \frac{\lfloor \frac{n}{2} \rfloor - (j-1)}{n - i_j + 1}$$

With this, the above probability becomes

$$\begin{aligned}
&\prod_{j=1}^{\lfloor \frac{n}{2} \rfloor} \left[\left(\prod_{k=i_{j-1}+1}^{i_j - 1} \left(1 - \frac{\lfloor \frac{n}{2} \rfloor - (j-1)}{n-k+1} \right) \right) \cdot \frac{\lfloor \frac{n}{2} \rfloor - (j-1)}{n - i_j + 1} \right] \\
&= \prod_{j=1}^{\lfloor \frac{n}{2} \rfloor} \left[\left(\prod_{k=i_{j-1}+1}^{i_j - 1} \left(\frac{n - \lfloor \frac{n}{2} \rfloor - (k-j)}{n-k+1} \right) \right) \cdot \frac{\lfloor \frac{n}{2} \rfloor - (j-1)}{n - i_j + 1} \right] \\
&= \frac{\prod_{j=1}^{i_{\lfloor \frac{n}{2} \rfloor} - \lfloor \frac{n}{2} \rfloor} \left(n - \lfloor \frac{n}{2} \rfloor - (j-1) \right) \cdot \prod_{j=1}^{\lfloor \frac{n}{2} \rfloor} \left(\lfloor \frac{n}{2} \rfloor - (j-1) \right)}{\prod_{j=1}^{i_{\lfloor \frac{n}{2} \rfloor}} (n-j+1)} \\
&= \frac{\prod_{j=1}^{n - \lfloor \frac{n}{2} \rfloor} \left(n - \lfloor \frac{n}{2} \rfloor - (j-1) \right) \cdot \lfloor \frac{n}{2} \rfloor !}{\prod_{j=1}^{n} (n-j+1)} \\
&= \frac{\left(n - \lfloor \frac{n}{2} \rfloor \right) ! \, \lfloor \frac{n}{2} \rfloor !}{n!} \\
&= \frac{\lfloor \frac{n}{2} \rfloor ! \, \lceil \frac{n}{2} \rceil !}{n!} \\
&= \frac{1}{\binom{n}{\lfloor \frac{n}{2} \rfloor}}
\end{aligned}$$

Therefore,

$$\Pr[S = T] = \frac{1}{\binom{n}{\lfloor \frac{n}{2} \rfloor}}$$

as required.

Note: A drawback of this solution, other than its implementation and runtime analysis becoming rather involved, is that it makes part (b) a lot harder. This is because the X_es in this solution are not pairwise independent and hence computing the variance of $\textsc{val}(S)$ becomes non-trivial, which in turn makes using Chebyshev's inequality difficult.

Common mistakes:

- Some students who used the alternative solution claimed that the probability that e is a crossing edge is precisely $\frac{1}{2}$.
- Some students considered a random vertex and put it in S and its neighbors in $V \setminus S$. This algorithm doesn't have $\mathbb{E}[\textsc{val}(S)] \geq \frac{m}{2}$, e.g., consider a complete graph G.

(b) [6 points] Design an $O((n+m)\log n)$ time Monte Carlo algorithm which given G outputs a cut $(S, V \setminus S)$ of G with $\textsc{val}(S) \geq \frac{m}{2} - \sqrt{m}$ with probability at least $1 - \frac{1}{m^2}$. Prove the correctness and runtime of your algorithm.

Hint: *Try using Chebyshev's inequality.*

Solution: The idea is to repeat the algorithm from part (a) several times.

We use the algorithm from part (a) to obtain a cut $(S, V \setminus S)$ and then check whether $\textsc{val}(S) > \frac{m}{2} - \sqrt{m}$. If so, we output it, if not we try again. Clearly, each try takes $O(n+m)$ time.

We would like to show that the chance of a cut as selected by the algorithm from part (a) having a value smaller than $\frac{m}{2} - \sqrt{m}$ is small. Notice that the X_e as defined earlier are pairwise independent. This is because any two edges can share at most one endpoint. Thus, even given how the endpoints of one edge were assigned, the other edge crosses the partition with probability $\frac{1}{2}$. Therefore,

$$\text{Var}[\textsc{val}(S)] = \sum_{e \in E} \text{Var}[X_e]$$

We have

$$\text{Var}[X_e] = \mathbb{E}[X_e^2] - (\mathbb{E}[X_e])^2 = \mathbb{E}[X_e] - (\mathbb{E}[X_e])^2 = \frac{1}{2} - \frac{1}{4} = \frac{1}{4}$$

and hence $\text{Var}[\textsc{val}(S)] = \frac{m}{4}$. Using Chebyshev's inequality,

$$\Pr\left[\left|\textsc{val}(S) - \frac{m}{2}\right| \geq \sqrt{m}\right] \leq \frac{\frac{m}{4}}{(\sqrt{m})^2} = \frac{1}{4}$$

Thus, with probability at least $\frac{3}{4}$, $\textsc{val}(S) \in (\frac{m}{2} - \sqrt{m}, \frac{m}{2} + \sqrt{m})$, i.e., $\textsc{val}(S) > \frac{m}{2} - \sqrt{m}$. Suppose we try at most $\log m$ times before we give up and output an arbitrary cut. The probability that every one of those attempts failed in producing a cut with value $> \frac{m}{2} - \sqrt{m}$ is at most $\left(\frac{1}{4}\right)^{\log m} = \frac{1}{m^2}$. Therefore, after $\log m = O(\log n^2) = O(\log n)$ attempts, we would have found a cut with value $> \frac{m}{2} - \sqrt{m}$ with probability at least $1 - \frac{1}{m^2}$. The total runtime of our algorithm is therefore $O((n+m)\log n)$.

Common mistakes:

- Many students did not show that the edges' indicator variables are pairwise independent, which is critical for the claim that $\text{Var}[\textsc{val}(S)] = \sum_{e \in E} \text{Var}[X_e]$.
- For students who used the alternative solution to part (a), the X_es are no longer pairwise independent. To correctly compute or at least upper bound $\text{Var}[\textsc{val}(S)]$, one must technically also account for the covariances between the X_es.

Problem 5. [15 points] **CRAZYSORT** (3 parts)
In this problem, we will be working with a data structure $D = (L, n)$ that consists of a list L of positive integers and a number n which is meant to be equal to the length of the list L. The integers in L need not be distinct.

Initially, $D_0 = (L_\emptyset, 0)$, where $L_\emptyset$ is the empty list. Our data structure D supports two operations:

- INSERT(x): Given a positive integer x, we add x to the beginning of L and update n to $n+1$. This has a cost of 1.
- CRAZYSORT$()$: We check if the first element x of L is $> \frac{n+1}{2}$. This has a cost of 1. If $x > \frac{n+1}{2}$, we do nothing. Otherwise, we
  - delete the first x elements of L and update n to $n - x$ (this has a cost of x), and
  - sort the next $t = \left\lfloor \frac{x}{2} \right\rfloor$ elements of L in non-increasing order (this has a cost of $2t \ln t$).

Note: *For $t = 0$, we define $2t \ln t = 0$.*

Define the potential function

$$\Phi(D) = 2 \cdot |L| \cdot \ln(|L| + 1)$$

(a) [2 points] Prove that Φ is a valid potential function.

Solution: We have $\Phi(D_0) = 2 \cdot 0 \cdot \ln(0+1) = 0$. Furthermore, $\ln(|L|+1) \geq \ln 1 = 0$ and hence $\Phi \geq 0$. Hence Φ is a valid potential function.

Common mistakes:

- Some students stated that $\Phi > 0$, which is not necessarily true as in the case where the first operation is a call to CRAZYSORT$()$ and the size of the L remains unchanged.

(b) [5 points] Suppose that the ith operation, for $i \geq 1$, is an INSERT. Use Φ to show that the amortized cost $\hat{c}_i$ is $O(\log n)$.
Hint: *You may use the fact that $\left(1 + \frac{1}{x}\right)^x \leq e$ for all $x \geq 1$.*

Solution: By definition, the amortized cost $\hat{c}_{\text{INSERT}}$ of INSERT is given by

$$\hat{c}_{\text{INSERT}} = c_{\text{INSERT}} + \Delta\Phi_{\text{INSERT}}$$

In INSERT, given a positive integer x, we add x to the beginning of L and update n to $n+1$. This has a cost of 1. We have $c_{\text{INSERT}} = 1$ and

$$\begin{aligned}
\Delta\Phi_{\text{INSERT}} &= 2 \cdot (|L|+1)\ln(|L|+1+1) - 2 \cdot |L| \cdot \ln(|L|+1) \\
&= 2 \cdot |L| \cdot \ln\left(\frac{|L|+2}{|L|+1}\right) + 2 \cdot \ln(|L|+2) \\
&\leq 2 \cdot (|L|+1) \cdot \ln\left(1 + \frac{1}{|L|+1}\right) + 2 \cdot \ln(|L|+2) \\
&= 2 \cdot \ln\left(\left(1 + \frac{1}{|L|+1}\right)^{|L|+1}\right) + 2 \cdot \ln(|L|+2) \\
&\leq 2 + 2 \cdot \ln(|L|+2) \\
&= O(\log n)
\end{aligned}$$

since $n = |L|$.

Common mistakes:

- Many students made arithmetic errors in deriving the correct upper bound on the amortized cost.
- Some students assumed that $|L| \cdot \ln(|L|+2)$ and $|L| \cdot \ln(|L|+1)$ were approximately equal and therefore canceled them out in the expression. This is not a rigorous method of upper bounding the amortized cost, which is what was expected.
- Some students assumed that the first i operations were INSERT(x) and used the aggregate method over amortized costs to derive an average of the amortized cost.

(c) [8 points] Suppose that the ith operation, for $i > 1$, is a CRAZYSORT. Use Φ to show that the amortized cost $\hat{c}_i$ is $O(1)$.

Solution: By definition, the amortized cost $\hat{c}_{\text{CRAZYSORT}}$ of CRAZYSORT is given by

$$\hat{c}_{\text{CRAZYSORT}} = c_{\text{CRAZYSORT}} + \Delta\Phi_{\text{CRAZYSORT}}$$

In CRAZYSORT, we check if the first element x of L is $> \frac{n+1}{2}$. This has a cost of 1. If $x > \frac{n+1}{2}$, we do nothing. In this case, $c_{\text{CRAZYSORT}} = 1$ and $\Delta\Phi_{\text{CRAZYSORT}} = 0$, and so $\hat{c}_{\text{CRAZYSORT}} = O(1)$.

Suppose $x \leq \frac{n+1}{2}$. We then delete the first x elements of L and update n to $n - x$ (this has a cost of x), and sort the next $t = \left\lfloor \frac{x}{2} \right\rfloor$ elements of L in non-increasing order (this has a cost of $2t \ln t$). In this case, $c_{\text{CrazySort}} = x + 2t \ln t + 1$ and

$$\begin{aligned}
\Delta\Phi_{\text{CrazySort}} &= 2 \cdot (|L| - x) \ln(|L| - x + 1) - 2 \cdot |L| \cdot \ln(|L| + 1) \\
&= 2 \cdot |L| \cdot \ln\left(\frac{|L| - x + 1}{|L| + 1}\right) - 2x \cdot \ln(|L| - x + 1) \\
&\leq -2x \cdot \ln(|L| - x + 1)
\end{aligned}$$

Therefore

$$c_{\text{CrazySort}} + \Delta\Phi_{\text{CrazySort}} \leq x + 2t \ln t - 2x \cdot \ln(|L| - x + 1) + 1$$

Now, since $x \leq \frac{|L|+1}{2} < \frac{2(|L|+1)}{3}$, $|L| + 1 > \frac{3x}{2}$, i.e.,

$$|L| - x + 1 > \frac{x}{2} \geq t$$

Therefore

$$\begin{aligned}
c_{\text{CrazySort}} + \Delta\Phi_{\text{CrazySort}} &\leq x + 2t \ln t - 2x \ln t + 1 \\
&\leq x + 2 \cdot \frac{x}{2} \cdot \ln t - 2x \ln t + 1 \\
&= x(1 - \ln t) + 1
\end{aligned}$$

If $t \geq e$, then $1 - \ln t \leq 0$ and hence $\hat{c}_{\text{CrazySort}} = O(1)$. We are left with the case of $t < e$. Notice that $\Delta\Phi_{\text{CrazySort}} \leq 0$ always. If $t < e$, then $t = O(1)$, $x = O(1)$, and hence $c_{\text{CrazySort}} = O(1)$.
Thus, in all cases, $\hat{c}_{\text{CrazySort}} = O(1)$.

Alternative Solution: When $x \leq \frac{n+1}{2}$, there are many different algebraic manipulations that one could do to prove that $\hat{c}_{\text{CrazySort}} = O(1)$. For instance:

$$\begin{aligned}
&c_{\text{CrazySort}} + \Delta\Phi_{\text{CrazySort}} \\
&\quad = x + 2t \ln t + 1 + 2 \cdot (|L| - x) \ln(|L| - x + 1) - 2 \cdot |L| \cdot \ln(|L| + 1) \\
&\quad \leq x + x \ln t + 1 + 2 \cdot (|L| - x) \ln(|L| + 1) - 2 \cdot |L| \cdot \ln(|L| + 1) \\
&\quad = x + x \ln t + 1 - 2x \cdot \ln(|L| + 1) \\
&\quad < x \cdot \ln(|L| + 1) + x \cdot \ln(|L| + 1) + 1 - 2x \cdot \ln(|L| + 1) \\
&\quad = 1
\end{aligned}$$

The first inequality follows as $t = \left\lfloor \frac{x}{2} \right\rfloor \leq \frac{x}{2}$ and $x > 0$. For the last inequality, also note that $t \leq \frac{x}{2} \leq \frac{n+1}{4} < |L| + 1$. The last inequality assumes that $|L| \geq 2$, so $\ln(|L| + 1) > 1$. If $|L| \leq 1$, then $x \leq 1$ and $t \leq 1$, so $\hat{c}_{\text{CrazySort}} = O(1)$.

Common mistakes:

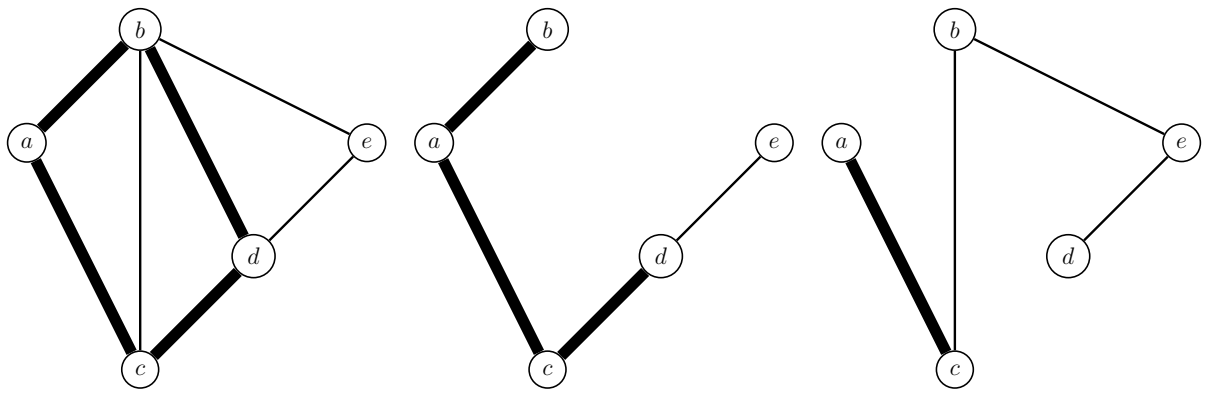
- Many students omitted the $+1$ term in $c_{\text{CrazySort}}$, the cost of checking if $x > \frac{n+1}{2}$.
- Some students tried to prove $\hat{c}_{\text{CrazySort}} = c_{\text{CrazySort}} + \Delta\Phi_{\text{CrazySort}} = O(1)$ by plugging in $x = \lfloor \frac{n+1}{2} \rfloor$, claiming that this is the worst case for x. This is incorrect without a proof since it's unclear whether $\hat{c}_{\text{CrazySort}}$ is an increasing or decreasing function in x (in fact, $\hat{c}_{\text{CrazySort}}$ is decreasing when $x \leq \frac{n+1}{2}$). This is because the $2 \cdot (|L| - x) \ln(|L| - x + 1)$ term decreases with x, but the other terms increase in x. It's not clear which effect is stronger.
- Many students made arithmetic errors in deriving the correct upper bound on the amortized cost.

Problem 6. [15 points] **Through Thick and Thin** (2 parts)
Let $G = (V, E)$ be a finite, simple, connected, undirected, and unweighted graph, where $n = |V|$ is odd. The edges of G have been categorized as being either thick or thin. You are also given two spanning trees T_{THICK} and T_{THIN} of G such that:

- T_{THICK} has $\geq \frac{n-1}{2}$ thick edges, and
- T_{THIN} has $\geq \frac{n-1}{2}$ thin edges.

The goal of this problem will be to find a spanning tree of G that has **exactly** $\frac{n-1}{2}$ thick edges and $\frac{n-1}{2}$ thin edges.

(a) [2 points] As a warm-up, consider the graph G with $n = 5$ below, along with two spanning trees T_{THICK} and T_{THIN} of G (thick edges, e.g., (a, b), are in **bold**).

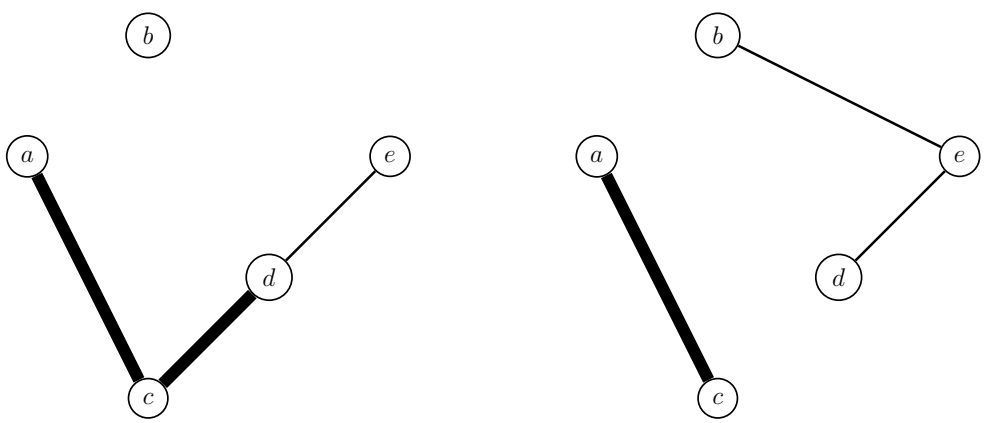


Warm-up graph G. Spanning Tree T_{THICK} of G. Spanning Tree T_{THIN} of G.

Complete the spanning trees T_1 and T_2 of G below by **drawing in missing edges** so that they each have exactly 2 thick edges and 2 thin edges.

No justification is needed (nor will be considered).

Note: *There may be more than one correct way of completing T_1 and T_2, but you only need to provide one correct way for each of them.*



Spanning Tree T_1 of G. Spanning Tree T_2 of G.

Solution: One can add in either one of the thin edges (b, c) or (b, e) to complete T_1, and either one of the thick edges (a, b) or (c, d) to complete T_2.

(b) [13 points] For a general graph G, given T_{THICK} and T_{THIN}, design an $O(n^2)$ time algorithm to find a spanning tree of G that has exactly $\frac{n-1}{2}$ thick edges and $\frac{n-1}{2}$ thin edges. Prove the correctness and runtime of your algorithm.

Hint: *Look at the example in part (a). Notice that $(b, c) \notin T_{\text{THICK}}$ and $(b, c) \in T_{\text{THIN}}$. What happens if we add (b, c) to T_{THICK}?*

Solution: First, if either T_{THICK} or T_{THIN} has exactly $\frac{n-1}{2}$ thick edges and $\frac{n-1}{2}$ thin edges, we are done. So, we assume that T_{THICK} has $> \frac{n-1}{2}$ thick edges and T_{THIN} has $> \frac{n-1}{2}$ thin edges. The idea, following from part (a), is to try and replace some thick edges in T_{THICK} with thin edges in T_{THIN} (or vice versa). We look at the thin edges in T_{THIN}. Let (u, v) be a thin edge in $T_{\text{THIN}} \setminus T_{\text{THICK}}$. If there is a thick edge (u', v') edge on the unique path connecting u and v in T_{THICK}, then we can add (u, v) to T_{THICK} and remove (u', v') from it. Note that T_{THICK} will still be a spanning tree, but now has one fewer thick edge than it did before. The goal then would be to repeatedly perform these swaps until T_{THICK} has exactly $\frac{n-1}{2}$ thick edges, in which case we are done.

What remains to be shown is that we will be able to perform such a swap. Firstly, note that there must certainly exist thin edges in $T_{\text{THIN}} \setminus T_{\text{THICK}}$. This is because T_{THIN} has more thin edges than T_{THICK}. Next, we claim that there must exist a thin edge (u, v) in $T_{\text{THIN}} \setminus T_{\text{THICK}}$ such that there is a thick edge (u', v') edge on the unique path connecting u and v in T_{THICK}. If not, consider the subgraph F of T_{THIN} induced by its thin edges. This subgraph is a forest consisting of $> \frac{n-1}{2}$ edges. Also consider the subgraph F' of T_{THICK} induced by its thin edges. If vertices u, v are connected in F, then they are connected in F' as well. Since F is a forest, F' must have at least as many edges as F. This means that T_{THICK} has $> \frac{n-1}{2}$ thin edges, which is not possible as it has $> \frac{n-1}{2}$ thick edges.

Thus, as long as T_{THICK} has $> \frac{n-1}{2}$ thick edges, we will be able to perform such a swap. We can use the argument above to find the edges (u, v) and (u', v') as well. We perform a DFS on both F and F'. For each edge (u, v) in $F \setminus F'$, we check to see whether u and v are connected in F'. By our previous argument, there will be a pair that is not connected in F'. We then look at the path connecting u and v in T_{THICK} to find a thick edge (u', v') on the unique path connecting u and v in T_{THICK}. Finally, we add (u, v) to T_{THICK} and remove (u', v') from it. Overall, a swap can thus be performed in $O(n)$ time. Since we will only have to do at most $\frac{n-1}{2} = O(n)$ swaps, our algorithm runs in $O(n^2)$ time, as required.

Alternative Solution 1: Another way of looking at the problem is to try and transform T_{THICK} into T_{THIN} and notice that along the way, we will find a spanning tree that has exactly $\frac{n-1}{2}$ thick edges and $\frac{n-1}{2}$ thin edges. Let $e_1, \ldots, e_{n-1}$ denote the edges in T_{THIN}. Let T be initially set to be T_{THIN}. For each $i \in \{1, \ldots, n-1\}$ in order, if $e_i \notin T_{\text{THICK}}$, we add e_i to T. This will create a cycle in T. We claim that this cycle must contain an edge $e_i' \notin T_{\text{THIN}}$, as otherwise we have found a cycle in T_{THIN}. We remove e_i' from T so that T remains a spanning tree. At the end of this process, by definition, $T = T_{\text{THICK}}$.

Let N_i denote the number of thin edges in T after the ith step, and let N_0 denote the number of thin edges in T_{THICK}. Note that $N_i = N_{i-1} \pm 1$. But $N_0 < \frac{n-1}{2}$ and $N_{n-1} > \frac{n-1}{2}$. Therefore, there must have existed an $i^* \in \{1, \ldots, n-2\}$ such that T, after i^* steps, is a spanning tree that has exactly $\frac{n-1}{2}$ thick edges and $\frac{n-1}{2}$ thin edges. Therefore, if we keep track of N_i, we can simply output T whenever $N_i = \frac{n-1}{2}$.

N_0 can be computed in $O(n)$ time using a linear scan. Each step of the algorithm involves finding a cycle which can be done in $O(n)$ time using a DFS. Looking at the edges added and removed at each step, we can compute N_i from N_{i-1} in $O(1)$ time. Thus each step can be performed in $O(n)$ time and the algorithm runs in $O(n^2)$ time, as required.

Alternative Solution 2: Similar to the first solution, we can replace thick edges in T_{THICK} with thin edges one at a time, conserving the property that it is a spanning tree, until we have exactly $\frac{n-1}{2}$ thick and exactly $\frac{n-1}{2}$ thin edges. However, instead of looking for such a thin edge one at a time, we can just iterate through all thin edges in $A := T_{\text{THIN}} \backslash T_{\text{THICK}}$. In particular, we let $T = T_{\text{THICK}}$ and then loop through the edges in $A := T_{\text{THIN}} \backslash T$. For each edge $e \in A$, we add e to T and find the cycle it induces, C_e^T. If C_e^T contains a thick edge, we delete this thick edge from T. Else, we remove e again from T. If after any "edge swap" T has exactly $\frac{n-1}{2}$ edges, we can halt and return T.

It is immediate that T will always be a spanning tree. What remains to show is that at some point T will have $\frac{n-1}{2}$ thin edges. First, note that for any edge e and tree T, C_e^T can only contain a thick edge if $C_e^{T'}$ did for all trees T' that previously appeared. We prove the contra-positive: Assume that $C_e^{T'}$ contains only thin edges. Since only thick edges get deleted, all these edges are still present in T. Since T is a spanning tree, we know that C_e^T is unique, and hence must be equal to $C_e^{T'}$, which implies that C_e^T contains only thin edges. Therefore, if C_e^T contains a thick edge, then $C_e^{T'}$ did for all trees T' that previously appeared. Equivalently, if $C_e^{T'}$ does not contain a thick edge, then neither will C_e^T for all trees T that will appear later.

To finish the proof of the algorithm, note that (as argued in solution 1) as long as T has $> \frac{n-1}{2}$ thick edges, there will be an edge $e \in A$ such that there is a thick edge in C_e^T. We now argue that this edge e cannot be a previously considered edge e'. This is because a previously considered edge e' either got added to T (and is hence no longer in A) or must be such that $C_{e'}^{T'}$ didn't contain a thick edge for some earlier tree T'. In the latter case, we know that $C_{e'}^T$ doesn't contain a thick edge either from our previous argument.

Therefore, we know that as T has $> \frac{n-1}{2}$ thick edges, there exists an edge $e \in A$ (that has not yet been considered) for which C_e^T contains a thick edge. Thus, we know that our algorithm will replace a thick edge in T with a thin edge from A until it ends up with a spanning tree with exactly $\frac{n-1}{2}$ thick and exactly $\frac{n-1}{2}$ thin edges.

We finally now discuss the runtime of our algorithm. We know that $|A| = O(n)$ since $|T_{\text{THIN}}| = n - 1 = O(n)$. For every edge $e \in A$, we can find the cycle C_e^T by running a DFS on $T \cup \{e\}$ in $O(n)$ time. Then, we can find and delete a thick edge from C_e^T if it exists, or delete e if not, in $O(n)$ time. Thus, the whole algorithm takes $O(n^2)$ time.

Common mistakes:

- Some students tried to build a tree by alternating between picking a thin edge and a thick edge. These approaches do not work since the algorithm may end up in a situation where the same number of thick and thin edges have been picked up to some point, but only thick (or only thin) edges are left to make the set of edges selected a spanning tree.
- Some students tried to start from T_{THICK} and then reduce the number of thick edges by finding a vertex that has both a thick edge currently in T_{THICK} and a thin edge currently not in T_{THICK}, and then deleting the thick edge and adding the thin edge. This doesn't work since the new set of edges in not guaranteed to be a tree.
- Many students claimed that when adding a thin edge to T_{THICK}, the resulting cycle will *always* contain a thick edge. This is incorrect.

Problem 7. [30 points] **One May Wonder** (5 parts)
The definition of universal hashing always seems like a curious one at first sight. Recall that a hash family $\mathcal{H} = \{h : \mathcal{U} \to M\}$ is universal if for all $k_1, k_2 \in \mathcal{U}$ where $k_1 \neq k_2$,

$$\Pr_{h \in_R \mathcal{H}}[h(k_1) = h(k_2)] \leq \frac{1}{m}$$

where $|M| = m$. One may wonder, why $\leq$ and not $=$?

To settle this question, we have part (a).

(a) [6 points] Construct a hash family $\mathcal{H} = \{h : \mathcal{U} \to M\}$ such that for all $k_1, k_2 \in \mathcal{U}$ where $k_1 \neq k_2$,

$$\Pr_{h \in_R \mathcal{H}}[h(k_1) = h(k_2)] < \frac{1}{m}$$

where $|M| = m$ and $|\mathcal{U}| > m$.

Note: *You are free to pick whatever $\mathcal{U}$ and M you like, subject to $|\mathcal{U}| > |M|$.*

Solution: There are many such hash families. One of them with $\mathcal{U} = \{a, b, c, d\}$, $m = 2$, $M = \{0, 1\}$, and $\mathcal{H} = \{h_1, h_2, h_3\}$ is as below.

	a	b	c	d
h_1	0	0	1	1
h_2	0	1	0	1
h_3	0	1	1	0

One can exhaustively check that for all $k_1, k_2 \in \mathcal{U}$ where $k_1 \neq k_2$,

$$\Pr_{h \in_R \mathcal{H}}[h(k_1) = h(k_2)] = \frac{1}{3} < \frac{1}{2} = \frac{1}{m}$$

Common mistakes:

- Some students proposed hash families with $|\mathcal{U}| \leq |M|$, under which one can trivially avoid all collisions.
- Some students proposed to use "random" hash functions to construct a hash family. Note that by definition, a hash function in a hash family must be deterministic.

We now know that collision probabilities can be $< \frac{1}{m}$, so one may wonder, how low can we go? To settle this question, we have parts (b) and (c).

(b) [6 points] Let $\mathcal{H} = \{h : \mathcal{U} \to M\}$ be a hash family with $|\mathcal{U}| = u$ and $|M| = m$. Let $h \in \mathcal{H}$ be a function from the family and define

$$C_h = |\{\{k_1, k_2\} : k_1, k_2 \in \mathcal{U}, k_1 \neq k_2, h(k_1) = h(k_2)\}|$$

denoting the number of colliding pairs under h. Prove that $C_h \geq \frac{u}{2m}(u - m)$.
Hint: *Suppose $t_1, \ldots, t_m \geq 0$ are non-negative integers such that $\sum_{i=1}^m t_i = u$, then we have $\sum_{i=1}^m t_i^2 \geq \frac{u^2}{m}$. You may use this inequality without proving it.*

Solution: Let $M = \{y_1, \ldots, y_m\}$. For each $1 \leq i \leq m$, let

$$t_i = |\{k : k \in \mathcal{U}, h(k) = y_i\}|$$

By definition, we have

$$\begin{aligned} C_h &= \sum_{i=1}^m \binom{t_i}{2} \\ &= \sum_{i=1}^m \frac{t_i(t_i - 1)}{2} \\ &= \frac{1}{2}\left(\sum_{i=1}^m t_i^2 - \sum_{i=1}^m t_i\right) \\ &\geq \frac{1}{2}\left(\frac{u^2}{m} - u\right) \\ &= \frac{u}{2m}(u - m) \end{aligned}$$

where we have used the fact from the hint since by definition, $t_1, \ldots, t_m \geq 0$ are non-negative integers such that $\sum_{i=1}^m t_i = u$.

Common mistakes:

- Some students assumed that the hash family was universal and tried to use the bound $\Pr_{h \in_R \mathcal{H}}[h(k_1) = h(k_2)] \leq \frac{1}{m}$.
- Some students tried to define t_i to be the number of collisions or an indicator for collisions in slot i and ran into issues trying to apply the hint.

(c) [6 points] Let $\mathcal{H} = \{h : \mathcal{U} \to M\}$ be a hash family with $|\mathcal{U}| = u$ and $|M| = m$. Prove that there exist $k_1, k_2 \in \mathcal{U}$, with $k_1 \neq k_2$, such that

$$\Pr_{h \in_R \mathcal{H}}[h(k_1) = h(k_2)] \geq \frac{1}{m}\left(\frac{u - m}{u - 1}\right)$$

Note: *If $u \gg m$, the bound is then essentially just $\frac{1}{m}$, which is how low we can go!*

Solution: Suppose we sample a random pair $k_1, k_2 \in \mathcal{U}$, with $k_1 \neq k_2$, and a random hash function h. Let X be the indicator random variable corresponding to $h(k_1) = h(k_2)$. We have

$$\begin{aligned}
\mathbb{E}[X] &= \Pr[X = 1] \\
&= \Pr_{k_1,k_2,h}[h(k_1) = h(k_2)] \\
&= \frac{\sum_{h \in \mathcal{H}} C_h}{\binom{u}{2} \cdot |\mathcal{H}|} \\
&\geq \frac{\frac{1}{2}\left(\frac{u^2}{m} - u\right) \cdot |\mathcal{H}|}{\frac{u(u-1)}{2} \cdot |\mathcal{H}|} \\
&= \frac{1}{m}\left(\frac{u-m}{u-1}\right)
\end{aligned}$$

Now, note that

$$\mathbb{E}[X] = \mathbb{E}_{k_1,k_2}\left[\Pr_{h \in_R \mathcal{H}}[h(k_1) = h(k_2)]\right]$$

Since $\mathbb{E}[X] \geq \frac{1}{m}\left(\frac{u-m}{u-1}\right)$, there must exist $k_1, k_2 \in \mathcal{U}$, with $k_1 \neq k_2$, such that

$$\Pr_{h \in_R \mathcal{H}}[h(k_1) = h(k_2)] \geq \frac{1}{m}\left(\frac{u-m}{u-1}\right)$$

Alternative Solution: There were several different ways to formulate the above solution:

- Some students used a counting argument, by using the bound from part (b) to lower bound the total number of collisions across all triples (h, k_i, k_j), then using a pigeonhole argument to show that there exists a pair of keys participating in at least $|\mathcal{H}| \cdot \frac{1}{m}\left(\frac{u-m}{u-1}\right)$ collisions.
- Some students did a proof by contradiction, by supposing that for all $k_1 \neq k_2$, $\Pr_{h \in_R \mathcal{H}}[h(k_1) = h(k_2)] < \frac{1}{m}\left(\frac{u-m}{u-1}\right)$, using that to find a strict upper bound on the expected number of collisions, and using the fact that $C_h \geq \frac{u}{2m}(u-m)$ to find a lower bound on the expected number of collisions to reach a contradiction.

Common mistakes:

- Some students lower bounded $\Pr_{k_1,k_2,h}[h(k_1) = h(k_2)]$ or $\Pr_{k_1,k_2}[h(k_1) = h(k_2)]$ for a fixed h.
- No points were deducted for this, but some students cited union bound instead of the definition of expectation or the linearity of expectation (depending on their presentation), and some students had unclear notation regarding what space they were taking each probability or expectation with respect to.

We now know that our definition of universal hashing is essentially as good as can be!

Let us turn to constructing universal hash families in parts (d) and (e).

Assume that a hash family has distinct hash functions, i.e., has no duplicate functions.

(d) [6 points] Let $\mathcal{H}_1 = \{h_1 : \mathcal{U} \to M\}$ and $\mathcal{H}_2 = \{h_2 : \mathcal{U} \to M\}$ be two universal hash families with $|\mathcal{U}| > |M|$. Define

$$\mathcal{H} = \{h_1 \| h_2 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}$$

where $\|$ denotes concatenation, i.e., $(h_1 \| h_2)(k) = (h_1(k), h_2(k))$. Note that $\mathcal{H}$ is a family of hash functions mapping $\mathcal{U}$ to $M \times M$. One may wonder, is $\mathcal{H}$ universal? If yes, prove it. If no, provide a counterexample.

Solution: Yes. First, we note that $\mathcal{H}$ has $|\mathcal{H}_1| \cdot |\mathcal{H}_2|$ distinct functions. If not, suppose $h_1 \| h_2 = h_1' \| h_2'$ for some $h_1, h_1' \in \mathcal{H}_1$ and $h_2, h_2' \in \mathcal{H}_2$. This must mean that $h_1 = h_1'$ and $h_2 = h_2'$, and thus either $\mathcal{H}_1$ or $\mathcal{H}_2$ contains duplicate functions, which contradicts the given assumption.

Consider any $k_1, k_2 \in \mathcal{U}$ where $k_1 \neq k_2$. Let $h = h_1 \| h_2$. In order for $h(k_1) = h(k_2)$, we must have $h_1(k_1) = h_1(k_2)$ and $h_2(k_1) = h_2(k_2)$. In other words, k_1 and k_2 must collide under $h_1 \in \mathcal{H}_1$ and $h_2 \in \mathcal{H}_2$. Since $\mathcal{H}_1$ and $\mathcal{H}_2$ are universal hash families, we have

$$|\{h_1 \in \mathcal{H}_1 : h_1(k_1) = h_1(k_2)\}| \leq \frac{|\mathcal{H}_1|}{m}$$

and

$$|\{h_2 \in \mathcal{H}_2 : h_2(k_1) = h_2(k_2)\}| \leq \frac{|\mathcal{H}_2|}{m}$$

Therefore,

$$|\{h \in \mathcal{H} : h(k_1) = h(k_2)\}| \leq \frac{|\mathcal{H}_1| \cdot |\mathcal{H}_2|}{m^2}$$

i.e.,

$$\Pr_{h \in_R \mathcal{H}}[h(k_1) = h(k_2)] \leq \frac{1}{m^2} = \frac{1}{|M \times M|}$$

Common mistakes:

- Many students did not write that $\mathcal{H}$ contains $|\mathcal{H}_1| \cdot |\mathcal{H}_2|$ distinct hash functions, i.e., every concatenation produces a different hash function. This is needed to deduce that $h \in_R \mathcal{H}$ has the same probability mass function as $h_1 || h_2$ where $h_1 \in_R \mathcal{H}_1$ and $h_2 \in_R \mathcal{H}_2$ are chosen independently.
- Some students bounded the probability of collisions under $h \in_R \mathcal{H}$ by $\frac{1}{m}$. Since $|M \times M| = m^2$, it needs to be bounded by $\frac{1}{m^2}$ for $\mathcal{H}$ to be a universal hash family.

(e) [6 points] Let $\mathcal{H}_1 = \{h_1 : \mathcal{U} \to M_1\}$ and $\mathcal{H}_2 = \{h_2 : M_1 \to M\}$ be two universal hash families with $|\mathcal{U}| > |M_1| > |M|$. Define

$$\mathcal{H} = \{h_2 \circ h_1 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}$$

where $\circ$ denotes composition, i.e., $(h_2 \circ h_1)(k) = h_2(h_1(k))$. Note that $\mathcal{H}$ is a family of hash functions mapping $\mathcal{U}$ to M. One may wonder, is $\mathcal{H}$ universal? If yes, prove it. If no, provide a counterexample.

Note: *$\mathcal{H}$ contains the hash functions obtained by composing hash functions from $\mathcal{H}_1$ and $\mathcal{H}_2$, and then* ***removing any duplicates****.*

Solution: No. There are many counterexamples. One of them is as follows. Consider the hash families $\mathcal{H}_1 = \{h_{1,1}, h_{1,2}, h_{1,3}\}$ and $\mathcal{H}_2 = \{h_{2,1}, h_{2,2}\}$ as described below. Here, $\mathcal{U} = \{a, b, c, d\}$, $M_1 = \{0, 1, 2\}$, and $M = \{0, 1\}$.

One can exhaustively check that

$$\Pr_{h_1 \in_R \mathcal{H}_1}[h_1(a) = h_1(b)] = \Pr_{h_1 \in_R \mathcal{H}_1}[h_1(b) = h_1(c)] = \Pr_{h_1 \in_R \mathcal{H}_1}[h_1(c) = h_1(d)] = \frac{1}{3}$$

$$\Pr_{h_1 \in_R \mathcal{H}_1}[h_1(a) = h_1(c)] = \Pr_{h_1 \in_R \mathcal{H}_1}[h_1(a) = h_1(d)] = \Pr_{h_1 \in_R \mathcal{H}_1}[h_1(b) = h_1(d)] = 0$$

$$\Pr_{h_2 \in_R \mathcal{H}_2}[h_2(0) = h_2(1)] = \Pr_{h_2 \in_R \mathcal{H}_2}[h_2(1) = h_2(2)] = \frac{1}{2}$$

$$\Pr_{h_2 \in_R \mathcal{H}_2}[h_2(0) = h_2(2)] = 0$$

Therefore, $\mathcal{H}_1$ and $\mathcal{H}_2$ are two universal hash families with $|\mathcal{U}| > |M_1| > |M|$. However, $\mathcal{H} = \{h_2 \circ h_1 : h_1 \in \mathcal{H}_1, h_2 \in \mathcal{H}_2\}$ is not universal as

$$\Pr_{h \in_R \mathcal{H}}[h(c) = h(d)] = \frac{2}{3} > \frac{1}{2}$$

	a	b	c	d
$h_{1,1}$	0	0	1	2
$h_{1,2}$	2	0	0	1
$h_{1,3}$	1	2	0	0

	0	1	2
$h_{2,1}$	0	0	1
$h_{2,2}$	1	0	0

	a	b	c	d
$h_1 = h_{2,1} \circ h_{1,1}$	0	0	0	1
$h_2 = h_{2,1} \circ h_{1,2}$	1	0	0	0
$h_3 = h_{2,1} \circ h_{1,3}$	0	1	0	0
$h_4 = h_{2,2} \circ h_{1,1}$	1	1	0	0
$h_5 = h_{2,2} \circ h_{1,2}$	0	1	1	0
$h_6 = h_{2,2} \circ h_{1,3}$	0	0	1	1

Common mistakes:

- Some students gave correct counterexamples but did not account for duplicates when showing that $\mathcal{H}$ is not a universal hash family.
- Some students provided explanations relating to the probability that two keys collide under $h_1 \in_R \mathcal{H}_1$. However, since duplicates have to be removed from $\mathcal{H}$, the explanations may not work. Specific counterexamples are needed.
- Some students gave counterexamples that do not satisfy $|\mathcal{U}| > |M_1| > |M|$.
- Some students gave counterexamples that are not fully or consistently specified.
- Some students gave counterexamples in which $\mathcal{H}_1$ is not a universal hash family. Note that the range of hash functions in $\mathcal{H}_1$ is M_1, so the probability of collisions under $h_1 \in \mathcal{H}_1$ must be at most $\frac{1}{|M_1|}$ for $\mathcal{H}_1$ to be a universal hash family.
- Some students gave counterexamples which, upon accounting for duplicates, produce $\mathcal{H}$ which is a universal hash family, e.g., the set of all functions from $\mathcal{U}$ to M.